Initial Scenarios for Onpoint

TS-001 — Checkout Page Detection

- **Feature:** Checkout detection (extension)
- **Preconditions:** Extension loaded in browser; detection enabled; no interfering extensions
- **Steps:**
    1. Open `extension/test-pages/checkout-sample.html` in a Chromium-based browser with the extension unpacked
    2. Wait up to 5 seconds for the extension detector UI to appear
- **Expected Result:** Extension detects the page as a checkout and shows the detector UI (badge or suggestions)
- **Priority:** High
- **Test Data:** `extension/test-pages/checkout-sample.html`
- **Environment:** Chrome/Chromium (desktop), extension unpacked; backend optional
- **Traceability:** SPEC.md — checkout detection requirement
- **Notes / Automation hints:** For CI, load the extension unpacked in Playwright/Chromium and assert presence of detector DOM element within timeout.

TS-002 — Card Auto-Detection and Recommendation

- **Feature:** Card detection & recommendation
- **Preconditions:** Detector active; sample checkout page loaded; `onpoint-cc-rewards/database/cards.json` seeded or mocked
- **Steps:**
    1. Open `extension/test-pages/checkout-sample.html` with the extension active
    2. Ensure form fields that resemble a checkout (card number, expiry, merchant name) are present
    3. Observe recommended card UI element
- **Expected Result:** Extension presents recommended card(s) and highlights the top pick based on reward/merchant matching
- **Priority:** High
- **Test Data:** `onpoint-cc-rewards/database/cards.json`, `onpoint-cc-rewards/database/merchants.json`
- **Environment:** Browser with extension; backend mocked or `onpoint-cc-rewards/backend-api` running locally
- **Traceability:** `recommendationService.js`, SPEC.md — recommendation rules
- **Notes / Automation hints:** Mock backend endpoints to return deterministic card and merchant responses; assert DOM content for recommendation and

that selected card ID matches expected.

TS-003 — Purchase Completion Detection and Recording

- **Feature:** Purchase completion capture
- **Preconditions:** Extension installed and configured; purchase sample page available
- **Steps:**
  1. Open `extension/test-pages/purchase-completion-sample.html` with the extension active
  2. Trigger the simulated completion flow on the page if interactive (or load the page variant that represents a completed purchase)
  3. Observe network calls or local logs for a purchase completion event
- **Expected Result:** Extension emits a purchase completion event; the backend receives and records the transaction (or a local confirmation is shown)
- **Priority:** Medium
- **Test Data:** `extension/test-pages/purchase-completion-sample.html`
- **Environment:** Browser with extension; backend available (`onpoint-cc-rewards/backend-api`) for e2e validation or mocked for unit tests
- **Traceability:** `purchase-detect-core.js` , backend controllers under `onpoint-cc-rewards/backend-api/controllers`
- **Notes / Automation hints:** Validate the event payload fields (merchant id, amount, transaction id). Use Playwright network intercepts to assert the outbound request body.

Review & Next Steps

- Review scenarios with product and QA; confirm priorities and missing scenarios.
- After sign-off: convert each approved scenario into executable tests (Playwright or Jest). Suggested ordering:
  1. Create Playwright smoke tests for TS-001 and TS-002 using the `extension/test-pages` static pages.
  2. Add network-mocking fixtures for recommendation and backend APIs.
  3. Add TS-003 as an e2e test that runs against a local backend or a recorded mock.